ASSIGNMENT 7

Name: Rohit Kailas Deore
Roll No: 11
Batch: T1
Class: TE-IT

[1]:
```
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay
)

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

1. Load the Breast Cancer Dataset

[2]:
```
# This built-in dataset avoids Kaggle downloads and extra packages
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = pd.Series(data.target, name="diagnosis")

print("First five rows:")
display(X.head())
print("Dataset shape:", X.shape)
print("Target classes:", list(data.target_names))
```

```
First five rows:
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	\
0	17.99	10.38	122.80	1001.0	0.11840	
1	20.57	17.77	132.90	1326.0	0.08474	
2	19.69	21.25	130.00	1203.0	0.10960	
3	11.42	20.38	77.58	386.1	0.14250	
4	20.29	14.34	135.10	1297.0	0.10030	

	mean compactness	mean concavity	mean concave points	mean symmetry	\
0	0.27760	0.3001	0.14710	0.2419	
1	0.07864	0.0869	0.07017	0.1812	

```
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  …  worst radius  worst texture  worst perimeter  \
0                 0.07871  …         25.38          17.33           184.60
1                 0.05667  …         24.99          23.41           158.80
2                 0.05999  …         23.57          25.53           152.50
3                 0.09744  …         14.91          26.50            98.87
4                 0.05883  …         22.54          16.67           152.20

   worst area  worst smoothness  worst compactness  worst concavity  \
0      2019.0            0.1622             0.6656           0.7119
1      1956.0            0.1238             0.1866           0.2416
2      1709.0            0.1444             0.4245           0.4504
3       567.7            0.2098             0.8663           0.6869
4      1575.0            0.1374             0.2050           0.4000

   worst concave points  worst symmetry  worst fractal dimension
0                0.2654          0.4601                  0.11890
1                0.1860          0.2750                  0.08902
2                0.2430          0.3613                  0.08758
3                0.2575          0.6638                  0.17300
4                0.1625          0.2364                  0.07678

[5 rows x 30 columns]
```

```
Dataset shape: (569, 30)
Target classes: [np.str_('malignant'), np.str_('benign')]
```

2. Split the Dataset

[3]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (455, 30)
Testing data: (114, 30)
```

3. Feature Scaling

[4]:
```
# Fit the scaler on training data only to avoid data leakage
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
print("Feature scaling completed.")
```

```
Feature scaling completed.
```

4. Logistic Regression

[5]:
```
log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_scaled, y_train)

y_pred_log = log_model.predict(X_test_scaled)

print("Logistic Regression Classification Report:")
print(classification_report(y_test, y_pred_log, target_names=data.target_names))
```

```
Logistic Regression Classification Report:
              precision    recall  f1-score   support

   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

5. K-Nearest Neighbors (KNN)

[6]:
```
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train_scaled, y_train)

y_pred_knn = knn_model.predict(X_test_scaled)

print("KNN (k=5) Classification Report:")
print(classification_report(y_test, y_pred_knn, target_names=data.target_names))
```

```
KNN (k=5) Classification Report:
              precision    recall  f1-score   support

   malignant       0.95      0.93      0.94        42
      benign       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```

6. Compare Model Performance

[7]:
```
# Use weighted averages so results account for class sizes
results = pd.DataFrame({
    "Model": ["Logistic Regression", "KNN (k=5)"],
    "Accuracy": [
        accuracy_score(y_test, y_pred_log),
        accuracy_score(y_test, y_pred_knn)
    ],
    "Precision": [
        precision_score(y_test, y_pred_log, average="weighted", zero_division=0),
        precision_score(y_test, y_pred_knn, average="weighted", zero_division=0)
    ],
    "Recall": [
        recall_score(y_test, y_pred_log, average="weighted", zero_division=0),
        recall_score(y_test, y_pred_knn, average="weighted", zero_division=0)
    ],
    "F1 Score": [
        f1_score(y_test, y_pred_log, average="weighted", zero_division=0),
        f1_score(y_test, y_pred_knn, average="weighted", zero_division=0)
    ]
})

display(results.round(4))

best_model = results.loc[results["Accuracy"].idxmax(), "Model"]
print("Better model based on accuracy:", best_model)
```

```
                Model  Accuracy  Precision  Recall  F1 Score
0  Logistic Regression    0.9825     0.9825  0.9825    0.9825
1            KNN (k=5)    0.9561     0.9561  0.9561    0.9560

Better model based on accuracy: Logistic Regression
```

7. Confusion Matrices

[8]:
```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

ConfusionMatrixDisplay(
    confusion_matrix(y_test, y_pred_log),
    display_labels=data.target_names
).plot(ax=axes[0], colorbar=False)
axes[0].set_title("Logistic Regression")

ConfusionMatrixDisplay(
    confusion_matrix(y_test, y_pred_knn),
    display_labels=data.target_names
).plot(ax=axes[1], colorbar=False)
```

```
axes[1].set_title("KNN (k=5)")

plt.tight_layout()
plt.show()
```



Conclusion

Logistic Regression and K-Nearest Neighbors were implemented using the Breast Cancer Wisconsin dataset. The features were scaled using training data only. Both models were evaluated using accuracy, precision, recall, F1-score, and confusion matrices. The comparison table shows which model performed better on the test data.